

Agentic RAG | Assignment 3

Evaluation Report

RAG Architect 2026 | DataSense

Candidate	Roopmathi Gunna
GitHub	github.com/SQLicious/Agentic_RAG_Assignment_3
Evaluator	Satvik, DataSense
Base Score	176 / 200
Bonus Score	45 / 60
Final Score	221 / 260
Grade	A

Score Overview

Category	Max	Score	%
1. Document Ingestion	15	15	100%
2. Adaptive RAG Router	20	19	95%
3. Corrective RAG Document Grader	20	18	90%
4. Corrective RAG Rewriter & Fallback	30	28	93%
5. Self-RAG Hallucination & Quality	25	23	92%
6. Graph Assembly & Loop Control	30	29	97%
7. Code Quality	30	24	80%
8. Evaluation	30	20	67%
Base Total	200	176	88%
Bonus: Advanced Chunking (+10)	10	10	
Bonus: Semantic Cache (+10)	10	10	
Bonus: 3-Tier Router (+10)	10	10	
Bonus: Architecture Diagram (+15)	15	15	
Bonus: Custom Implementation (+15)	15	0	
Bonus Total	60	45	75%
Grand Total	260	221	85%

Detailed Feedback

1. Document Ingestion | 15 / 15

#	Criterion	Max	Score	Notes
1.1	Loaded 3+ documents	3	3	Full marks
1.2	Chunking strategy implemented & justified	4	4	Full marks
1.3	Embeddings stored in vector DB	4	4	Full marks
1.4	Metadata attached to chunks	2	2	Full marks
1.5	Ingestion runs end-to-end	2	2	Full marks

Roopmathi, you nailed this section. You loaded six documents into the pipeline: one PDF (the Claude Certified Architect exam guide) and five markdown snapshots covering Claude 101, Building with the Claude API, Claude Code in Action, Introduction to Claude Cowork, and Introduction to Model Context Protocol. That comfortably clears the three-document minimum, and mixing PDF + markdown formats shows your loader handles both correctly.

Your default chunking strategy is hierarchical parent-child chunking, and it's done properly. In `chunker.py`, you split documents into heading-aware sections using a custom heading detector (`_looks_like_heading`), then split those into parent chunks (1800 chars, 200 overlap) and child chunks (700 chars, 120 overlap). Children get embedded and indexed in ChromaDB; at retrieval time, matched children map back to their parent via `parent_store.json`, giving the generator a wider context window. Your README explains the rationale well: smaller chunks improve retrieval precision, parent rehydration preserves context for generation.

Metadata is rich. Each chunk carries `source_type`, `file_name`, `section_title`, `section_index`, `parent_id`, `chunking_strategy`, and `chunk_level`. That feeds directly into your citation system downstream. The indexer also resets collections before re-ingestion, which avoids the duplicate-chunk problem I see in many other submissions. Nothing to dock here.

2. Adaptive RAG Router | 19 / 20

#	Criterion	Max	Score	Notes
2.1	Router classifies into 2+ tiers	6	6	Full marks
2.2	Uses structured output	5	5	Full marks
2.3	LLM-direct path demonstrated	4	4	Full marks
2.4	Web search path demonstrated	3	3	Full marks
2.5	Route decision logged	2	1	See feedback below

Your router works well. `route_question` in `router.py` uses GPT-4o-mini with a Pydantic model (`RouteDecision`) that returns both a route label and a reason explaining the decision. The router prompt is domain-aware and includes rules for when to use `vectorstore` versus `web_search` versus `llm_direct`. The `force_route` mechanism for decomposed sub-questions is a nice touch most submissions miss.

Your demo notebook covers all three branches convincingly, and your screenshots in `docs/screenshots/` confirm these work in CLI and UI modes.

The 1-point deduction: `route` and `route_reason` land in the result dictionary, but nothing gets printed to the console while the graph is running. When I execute the pipeline from a notebook, I see the final result, but I don't see a real-time line like "Router: vectorstore (question refers to exam guide content)" as execution progresses. A simple `print` or `logging.info` inside `route_question` would fix this and make debugging much easier.

3. Corrective RAG | Document Grader | 18 / 20

#	Criterion	Max	Score	Notes
3.1	Binary relevance grader per chunk	8	8	Full marks
3.2	Structured output	5	5	Full marks
3.3	Irrelevant docs filtered	4	4	Full marks
3.4	Grading decisions logged with reasoning	3	1	See feedback below

Your `grade_documents` function iterates over each chunk, sends it to the LLM alongside the question, and gets back a `BinaryGrade` Pydantic model with a "yes" or "no" score. Only "yes" chunks make it into `filtered_docs`. The prompt is more specific than what I've seen in most submissions: it tells the grader to reject chunks that are "irrelevant, too broad, or too weak" rather than just asking "is this relevant?" That's a meaningful distinction.

Same logging issue here. The grader populates `doc_relevance` and `relevant_doc_count` in the state, so the final result shows how many docs passed. But during execution there's no per-doc output showing which chunk got which grade. If you're debugging a run where the grader rejected everything, you'd have to manually add `print` statements to figure out what happened. A brief per-chunk log line would go a long way.

4. Corrective RAG | Rewriter & Fallback | 28 / 30

#	Criterion	Max	Score	Notes
4.1	Query rewriter implemented	8	8	Full marks
4.2	Rewritten query triggers fresh retrieval	6	6	Full marks
4.3	Fallback to web search	8	8	Full marks

4.4	Fallback to LLM parametric knowledge	4	2	See feedback below
4.5	Graceful degradation	4	4	Full marks

This is some of the best corrective flow work I've seen across submissions. Your rewriter doesn't just rephrase the query; it takes the original question, the previous retrieval query, AND a summary of the failed chunks with titles and previews, so the LLM has real context about what went wrong. That's a level of detail most people don't bother with.

Your web search fallback in fallback.py is genuinely sophisticated. It runs a two-pass Brave Search: first official Anthropic domains only, then broadening if nothing comes back. Results get scored with a trust model that boosts official domains (+100), penalizes Reddit/Medium (-60), and ranks by title relevance. Deduplication, top-3 selection, structured error messages. For a course assignment, this is production-level work.

Where you lost 2 points: the LLM parametric fallback isn't wired. When the web search itself errors out, your generator returns "Web search is currently unavailable...I could not verify the latest information right now." That's graceful, but it's not a parametric fallback. It doesn't try to answer the question from the model's own knowledge with a disclaimer. You already have generate_direct_answer working perfectly for Tier 0; routing to it as a last resort when all retrieval fails is probably 15 lines of code and would close this gap.

5. Self-RAG | Hallucination & Quality | 23 / 25

#	Criterion	Max	Score	Notes
5.1	Hallucination grader	8	8	Full marks
5.2	Answer quality grader	8	8	Full marks
5.3	Failed grading triggers retry	5	5	Full marks
5.4	Grading results logged	4	2	See feedback below

Both graders work correctly. grade_hallucination checks whether the answer is grounded in context; grade_answer_quality checks whether it addresses the question. Both use the BinaryGrade Pydantic model.

The retry loop is solid. When the hallucination grader fails, the pipeline checks the iteration counter, and if under the max (3), retries via web_search. When quality fails, same logic. If max iterations are hit, the pipeline terminates with whatever answer it has. I traced all conditional edges in graph.py and confirmed there are no paths that can loop forever.

One detail I liked: route_after_hallucination and route_after_quality both check for web_search_error and route to END if it's set. That prevents retrying when the Brave API itself is broken. Subtle edge case, and most submissions miss it entirely.

The 2-point deduction is the same logging theme. hallucination_grade and answer_quality_grade land in the final result, but during execution there's no real-time output. Adding in-flight log lines for these decisions would help a lot.

6. Graph Assembly & Loop Control | 29 / 30

#	Criterion	Max	Score	Notes
6.1	Full pipeline as graph/state machine	10	10	Full marks
6.2	Conditional edges based on grader outputs	6	6	Full marks
6.3	Max retry limit enforced	6	6	Full marks
6.4	All branches converge to END	4	4	Full marks
6.5	Clean state management	4	3	See feedback below

This is your strongest section and one of the best graph implementations I've reviewed. `build_graph` in `graph.py` constructs a complete LangGraph StateGraph with clearly named nodes and well-structured conditional edges. The graph covers the full spec: initialization, 3-way routing, retrieval, document grading, query rewriting with retry, web search fallback, generation, hallucination grading, quality grading, and iteration control. Every branch converges to END. I traced all possible paths and there are no orphan nodes or dead ends.

The edge functions are clean and readable. `route_after_doc_grading` implements the corrective decision tree. `route_after_hallucination` and `route_after_quality` handle Self-RAG retries with max-iteration enforcement. `retry_route` adds an extra layer: on retry it routes to `web_search` for vectorstore/web routes but to END for `llm_direct`, which makes sense since there's nothing to retry for a direct answer.

Your state is a proper TypedDict (GraphState in `state.py`) with all fields typed. Every field has a Literal or typed annotation. The 1-point deduction is minor: a few state fields like `web_search_error` get set inside nodes but aren't initialized in `initialize_state`. Initializing every field upfront would make the full state shape visible in one place.

7. Code Quality | 24 / 30

#	Criterion	Max	Score	Notes
7.1	Modular structure per spec	8	8	Full marks
7.2	Docstrings on functions	4	2	See feedback below
7.3	Error handling	6	4	See feedback below
7.4	No hardcoded API keys	4	4	Full marks
7.5	requirements.txt present & complete	4	3	See feedback below
7.6	README covers all required sections	4	3	See feedback below

Your repo structure goes beyond the spec. You have `src/ingestion/` with loader, chunker, and indexer; `src/nodes/` with all the pipeline nodes; `src/cache/` for semantic cache; `graph.py`, `state.py`,

api.py, app.py (FastAPI server), and a full React UI in ui/. The notebooks/ and evaluation/ directories are populated. Good organization.

Docstrings: you left points here. Your main pipeline functions (grade_documents, route_question, rewrite_query, etc.) have brief docstrings. But out of roughly 60 functions across the codebase, only about 18 have any documentation. The public API functions (run_single_question, run_adaptive_query) have basic docstrings but don't describe parameters or return types. That's a 2-point hit.

Error handling is uneven. fallback.py wraps the Brave API call in try/except and returns a structured error. But planner.py catches a bare Exception on line 540 and silently falls back to the heuristic plan. app.py also catches bare Exception. The core graph nodes (router, grader, generator) have no error handling; if the OpenAI API call fails, the pipeline crashes with an unhandled exception. In production you'd want specific exception types.

requirements.txt covers the dependencies but doesn't pin versions (except ragas). In a team setting, unpinned deps cause reproducibility problems.

Your README is comprehensive. The trade-offs table listing decision, rationale, and downside for each design choice is better than most. I docked 1 point because while you explain why you chose LangGraph, you don't discuss what alternatives like LlamaIndex or a custom state machine would have given you and why you rejected them.

8. Evaluation | 20 / 30

#	Criterion	Max	Score	Notes
8.1	3+ diverse test questions	8	8	Full marks
8.2	Metrics reported	8	6	See feedback below
8.3	Results table with breakdown	6	4	See feedback below
8.4	Written analysis	4	1	See feedback below
8.5	Source citations visible	4	1	See feedback below

You have the largest evaluation dataset I've seen across all submissions: 60 test questions organized across Tier 0 (20), Tier 1 (33), and Tier 2 (7). Each question has an expected tier, and you report observed-vs-expected matching (47 exact, 13 mismatches). The run_ragas_eval.py computes all four RAGAS metrics: answer relevancy, faithfulness, context precision, context recall. That's solid work on the test set and metric side.

Where you lost points: the per-tier breakdown tells a story you didn't write up. Your Tier 2 answer relevancy is 0.29 and 13 out of 20 expected Tier 2 questions got misclassified as Tier 1. That's a big planner failure rate. The findings section acknowledges it but doesn't dig into why. A couple of sentences explaining which planner regex patterns failed to trigger and what kind of questions slipped through would have turned raw data into an engineering insight.

The findings section is four bullet points. For a 30-point section, that's not enough. What did Tier 0 do well? Why is context recall at 0.31? Are there question types where the grader was too lenient? Which fallback paths triggered most? Your data has the answers. The analysis just wasn't written.

Source citations: your generator correctly builds citations from metadata and your `api.py` surfaces them in the result. But in the evaluation results themselves, citations don't appear in per-sample output. The `results.md` shows questions, tiers, routes, and metrics, but not the generated answers or their citations. Your notebook cells have no saved output (not executed before commit). The static HTML export is a good idea, but the assignment asks for citations to be visible in the demo output, and I can't verify them from what's in the repo.

This is the section where you left the most points on the table. 20 minutes of writing analysis and re-running the notebook with saved outputs could easily add 8-10 points here.

Bonus Features

Advanced Chunking | 10 / 10

You implemented both recursive and hierarchical parent-child chunking, and ran a proper ablation study. `run_chunking_ablation.py` runs all 60 questions against both strategies and reports four RAGAS metrics side by side. Hierarchical beats recursive on faithfulness (0.80 vs 0.68), context precision (0.53 vs 0.31), and answer relevancy (0.57 vs 0.53). Recursive edges out on context recall, which makes sense since smaller flat chunks cast a wider net. The `ablation_results.md` includes an interpretation note. Full marks.

Semantic Cache | 10 / 10

Your cache in `src/cache/semantic_cache.py` is well thought out. SQLite stores query embeddings and full result payloads. Cosine similarity against stored embeddings, configurable 0.92 threshold. Time-sensitive queries ("recent," "latest," "today") bypass the cache, preventing stale answers. The cache is tied to a corpus version fingerprint computed from file names, sizes, and modification timestamps, so it auto-invalidates when documents change. Cache hits genuinely skip the full pipeline and return the stored answer with citations. Full marks.

3-Tier Router | 10 / 10

All three tiers are implemented and working. `planner.py` classifies queries into Tier 0 (LLM-only), Tier 1 (single-route RAG), and Tier 2 (multi-hop with decomposition). The decomposition logic in the planner detects comparison patterns, alignment patterns, and mixed-retrieval intents. It also catches and collapses overlapping sub-questions back to single-pass. Tier 2 runs each sub-question through the full single-question graph independently, then merges answers via `merge.py`. `tiering.py` maps plan types to tier labels cleanly. All three tiers demonstrated in notebook and screenshots. Full marks.

Architecture Diagram | 15 / 15

Your diagram in `docs/Architecture_diagram.png` covers the complete pipeline from user query through semantic cache, planner, three-tier execution, all the way to final answer. Tiers are visually separated. Decision diamonds show branching logic correctly. The corrective flow (rewriter, retry, Brave fallback) and Self-RAG flow (hallucination check, quality check, retry) are both visible with dashed retry arrows. Color coding is consistent and the whole pipeline is followable without reading code. One of the best diagrams across all submissions. Full marks.

Custom Implementation | 0 / 15

You used LangGraph as your orchestration layer, so this bonus doesn't apply. Not a deduction; you just didn't attempt this one.

Overall Assessment

Summary

Roopmathi, this is a strong submission and one of the better ones I've reviewed this semester. You built a complete agentic RAG system that goes well past the baseline. All three RAG layers work: Adaptive routing with a 3-way classifier, Corrective grading with query rewriting and a two-pass web search fallback, and Self-RAG with hallucination and quality checks feeding a bounded retry loop. On top of that, you delivered four out of five bonus features: hierarchical chunking with an ablation study, a corpus-aware semantic cache with time-sensitivity bypass, a 3-tier routing system with multi-hop decomposition and answer merging, and a polished architecture diagram.

Where you fell short is mostly in two places. Your evaluation section has an impressive 60-question test set and all four RAGAS metrics, but the analysis section is four bullet points where it should be four paragraphs. The data is there; the interpretation isn't. And your code documentation could be tighter: many functions lack docstrings, error handling is uneven in places, and the notebook wasn't saved with outputs.

221 / 260, grade A. With maybe 30 more minutes of effort on the evaluation writeup, adding docstrings to your key functions, and wiring the LLM parametric fallback, this would be a 240+ submission.

What You Did Well

Your system design stands out. Most submissions build a flat pipeline. You built a layered architecture: a planner above the single-question graph, a semantic cache above the planner, and a 3-tier execution model routing queries to the right complexity level. Tier 2 decomposes a question into sub-questions, runs each through the full graph with Self-RAG grading, and merges answers. That's real multi-agent orchestration.

Your web search fallback is production-grade. Two-pass Brave Search (official domains first, broad second), a trust-scoring model with domain allow/deny lists, structured error handling. This is the kind of work that separates course projects from deployable systems.

The bonus features aren't bolt-ons. The semantic cache sits in front of the planner and genuinely skips the pipeline. The hierarchical chunking feeds parent-rehydrated context to the retriever. The 3-tier router drives the entire execution model. These earn bonus points because they're integrated, not just checked off.

What Needs Work

Write a proper evaluation analysis. You have 60 questions and four metrics per question. Your Tier 2 planner misfired on 13 of 20 expected multi-hop questions. Why? Context recall is 0.31 overall. What does that mean for answer quality? Where are the unfaithful responses coming from? Answer these questions in writing and your evaluation section jumps from 20 to 28.

Add runtime logging to your graph nodes. Every node should print its key decision: route chosen, docs graded relevant, rewritten query, hallucination/quality verdict. Right now the only way to see these is to inspect the result dict after the pipeline finishes. Real-time logging makes debugging, demoing, and evaluating much easier.

Wire the LLM parametric fallback. You already have `generate_direct_answer` working for Tier 0. Adding a final node that calls it with a disclaimer when both retrieval and web search fail would complete the fallback chain and pick up the last 2 points in section 4.

Production Readiness

Your pipeline is about 85% of the way to production-ready, which is unusually high for a course assignment. To close the gap I'd prioritize: structured logging with trace IDs and per-node latency tracking, the complete fallback chain so no query dead-ends, pinned dependency versions with a CI pipeline running your 60-question eval nightly, source citations surfaced inline in the generated text (not just in metadata), and prompt versioning so you can A/B test router/grader/generator prompt changes against your eval set.

Good work, Roopmathi. Keep building.

Satvik

DataSense